Lab 2.2 - Build the tester's stack (Testers)

Goal

Learn what goes where by building a stack for test work: a rule file for test code, a test-design agent that cannot write code, a skill with a template, and a read-only connection to the work items. Then prove each piece loads when it should, and only then.

You're done when

- ☐ The "where does it go?" table (step 1) is filled, with a one-line reason per row
- ☐ You proved the test rule loads for test files and does **not** load for production code
- ☐ You saw what happens when two rule files disagree, and fixed it
- ☐ The `test-designer` agent writes Given/When/Then but cannot edit files, and you proved it
- ☐ `/test-charter` (a skill) produces a charter from its template
- ☐ The agent read WI-1042 through a read-only MCP server, and did not invent acceptance criteria

Option A - Guided

1. Where does it go? (5 min, no AI). For each rule, pick its home: **AGENTS.md**, **copilot-instructions.md**, **path rule** (`*.instructions.md`), **prompt file**, **skill**, **custom agent**, **MCP server**, **hook**, or **nowhere**. More than one can be right. Write one line on why.

#	Rule	Home	Why
a	How to run the web tests and the e2e tests		
b	Test names: `Method_Scenario_Result`		
c	e2e selectors: `data-testid` only, never CSS classes		
d	Only synthetic data (SYN-xxxx studies) in tests and prompts		
e	"Write an exploratory charter" with our team template		
f	Test design must never change production code		
g	Read acceptance criteria from our board		
h	Block any test run against a production URL		
i	SYN-0002 kidney volume is 129.6 mL		

2. A path rule for test code (5 min). Create `.github/instructions/tests.instructions.md`:

```
---
applyTo: "src/OrganLens.Api.Tests/**,tests/e2e/**"
description: "Rules for automated tests"
---
- xUnit for .NET tests. Names: <Method>_<Scenario>_<Result>. One behaviour per test.
- Integration tests use ApiFactory (WebApplicationFactory). No real network, no real database.
- Playwright: select elements by data-testid only. No fixed waits (no waitForTimeout).
- Only synthetic data: SYN-xxxx study ids, "Test Patient NN" names.
- A test must be able to fail. No assertions that are always true.
```

3. Prove it loads, and only where it should (5 min). New chat, **Ask** mode, with `src/OrganLens.Api.Tests/Integration/StudiesEndpointTests.cs` open:

```
What rules must a new test in this file follow? Which rule files did you use to answer?
```

Expand **References** at the top of the answer. Is `tests.instructions.md` there? Now open `src/OrganLens.Api/Services/MeasurementService.cs`, new chat, same question. The test rule must **not** appear. If it does, check your `applyTo`.

4. Make two files disagree (5 min). Add to `AGENTS.md` (create it if needed):

```
- Test names: Should_<Result>_When_<Scenario>
```

Ask 3 times, each in a new chat, with the test file open:

```
Suggest a name for a test that checks GET /api/studies/SYN-9999 returns 404.
```

Write down what you got. Both rules were sent; the model chose. Delete one. Rule: **one home per rule.**

5. A test-design agent that cannot code (7 min). Create `.github/agents/test-designer.agent.md`:

```
---
name: test-designer
description: Designs test cases from acceptance criteria. Never writes or edits code.
tools: ['read', 'search']
---
You design tests. You never edit files.
Output Given/When/Then cases with a data table (synthetic SYN-xxxx data only).
Mark each case: unit, integration, e2e or manual, with one line on why.
If the requirement is unclear, list questions instead of guessing.
```

Pick **test-designer** in the agents dropdown and ask:

```
Design tests for: "GET /api/studies/{id} returns 404 for an unknown study".
Then save them to docs/notes/tests-404.md.
```

It should design the tests but fail to save them. Good.

> Tool names differ between versions. Type # in the chat box to see the exact names in yours.

6. A skill with a template (8 min). Create the folder `.github/skills/test-charter/` with two files. `SKILL.md`:

```
---
name: test-charter
description: Writes an exploratory test charter for a feature or risk. Use when asked for a
charter, an exploratory session, or a test session plan.
---
1. Read charter-template.md in this folder and fill it in.
2. Base the risks on the code: cite file + method for each.
3. Use only synthetic data (SYN-xxxx).
4. Keep it to one page.
```

`charter-template.md`:

```
# Charter: <title>
Explore <area> with <resources> to discover <information>.
Time box: 30 min
Risks (with code location):
Test ideas (5 max):
Data:
Notes / bugs / questions:
```

Run:

```
/test-charter the manual measurement form
```

Note: until you ran it, only the skill's name and description were in context. The template was read only when it was needed.

7. Read the work item (8 min). Create `.vscode/mcp.json`:

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "X-MCP-Readonly": "true" }
    }
  }
}
```

Click **Start**, sign in. Hover the **context-window control** before and after: how many tokens did the tools add? **Agent** mode, with **test-designer** selected:

```
Using the GitHub MCP server, read issue WI-1042 in CodeValue/training-philips. Quote it
exactly.
It has no acceptance criteria. Do not invent any.
List the questions a tester must ask the product owner before tests can be written.
```

Did it stick to questions, or did it slip in made-up criteria?

Option B - Challenge

Build the same stack without the snippets above, plus:

- Rule h ("never test against production") is too important to be a request. Write a hook in `.github/hooks/` that denies terminal commands containing your production host name. Read the hooks docs first. Which surfaces run it today?
- Add a handoff from `test-designer` to the default agent ("Implement these tests"), so the design and the coding are two separate, reviewed steps.

Stop and check

- Why is the charter a skill and not a rule in AGENTS.md?
- What is the difference between telling an agent "don't edit" and removing its edit tool?
- WI-1042 has no acceptance criteria. Who should write them: you, the developer, the product owner or the agent?

Before you leave

Commit the stack to `main`, so later labs can use it:

```
git add AGENTS.md .github .vscode docs
git commit -m "Tester stack: rules, test-designer agent, charter skill, MCP"
```